

AKSHAR AI

SUMMER INTERNSHIP REPORT

WEEK 5 | 20 JUNE 2026 – 26 JUNE 2026

Project Title: Pneumonia Detection System & Human Behavior Tracking System

DETAILS

Name	Yashas Raj R
Enrollment No.	24162121033
Branch	B.Tech – Computer Science and Engineering (Big Data Analytics)
Institute	ICT Ganpat University, Mehsana, Gujarat
Organisation	Akshar AI Ltd
Report Period	22 June 2026 – 27 June 2026 (Week 5)
Email	24162121033@gnu.ac.in

1. Introduction

Following the intense architectural research and strategic planning phase of Week 4, Week 5 represented a dramatic shift towards aggressive implementation and empirical experimentation. The internship continued to operate on dual tracks, demanding high-level focus on both the medical imaging pipeline (Pneumonia Detection System) and the computer vision module (Hand & Gesture Recognition).

For the Pneumonia Detection System, the primary directive was the execution of the dataset scaling strategy finalized in the previous week. The fundamental hypothesis being tested was that increasing the training volume from the initial 50-image dataset to incrementally larger datasets (200, 500, 700, and 1000 images) would mitigate the severe overfitting observed in early models and significantly enhance generalization capabilities. This phase required the meticulous retraining of the three core architectures—Custom CNN, MobileNetV2, and ResNet50—across these new data milestones. This effort involved substantial compute time, rigorous logging of performance metrics, and deep analytical comparisons to identify the optimal model architecture for eventual clinical deployment.

Concurrently, the Hand & Gesture Recognition Module transitioned from theoretical design to practical coding. Leveraging the insights gained regarding Google MediaPipe and OpenCV, the objective was to build the foundational Python pipeline capable of ingesting live webcam feeds, executing the MediaPipe palm detection and landmark regression models, and extracting the 21 3D spatial coordinates in real-time. This foundational code serves as the core

engine upon which complex gesture classification algorithms and standardized JSON output formatting will be built in subsequent weeks.

This comprehensive report details the rigorous technical activities, experimental executions, and analytical processes conducted throughout Week 5. It chronicles the day-by-day progression from writing the initial OpenCV bridging code to the final extraction and visualization of deep learning metrics, providing a transparent view of the implementation methodologies and the crucial insights derived from large-scale model training.

2. Weekly Objectives

The objectives for Week 5 were heavily weighted toward code execution, model training, and data analysis. The goals were designed to practically implement the theories and plans established during the research phase.

The primary objectives for Week 5 were defined as follows:

Computer Vision Implementation (Hand Tracking): Transition the MediaPipe architectural research into a functional Python script. Establish a robust OpenCV video capture loop that streams live webcam data with minimal latency.

Real-time Landmark Extraction: Successfully integrate the `mediapipe.solutions.hands` API into the video loop. Configure the API for high-confidence tracking and write the logic to extract and visualize the 21 3D hand landmarks superimposed on the live feed.

Dataset Preparation & Augmentation: Prepare the chest X-ray dataset for the scaling experiments. Ensure balanced classes (Normal vs. Pneumonia) for the 200, 500, 700, and 1000 image subsets, and apply appropriate data augmentation pipelines to maximize data utility.

Iterative Model Retraining (200 & 500 Images): Execute the retraining of the Custom CNN, MobileNetV2, and ResNet50 models on the 200-image and 500-image datasets. Monitor for convergence and log all hyperparameter configurations.

Iterative Model Retraining (700 & 1000 Images): Continue the scaling experiments by training all three architectures on the larger 700-image and 1000-image datasets. Observe the impact of high data volume on the performance gap between custom and pre-trained networks.

Empirical Metric Analysis: Consolidate the training logs (Accuracy, Validation Accuracy, Loss, Validation Loss) across all dataset sizes and model architectures. Perform a comparative analysis to identify the model that exhibits the highest generalization and lowest overfitting delta.

Documentation and Version Control: Maintain the rigorous documentation standards established in Week 4. Record all experimental results in the tracking matrix and commit the newly developed hand tracking source code and Jupyter notebooks to the Git repository.

Day-wise Objective Summary

1	20 Jun 2026	OpenCV setup & MediaPipe integration for live tracking	Functional Python script capturing and displaying live video
2	22 Jun 2026	Extracting 21 3D landmarks and drawing connecting topologies	Real-time visual overlay of the kinematic hand structure
3	23 Jun 2026	Dataset prep & Retraining CNN/MobileNet/ResNet on 200/500 images	Initial scaling results logged in the tracking matrix
4	24 Jun 2026	Retraining CNN/MobileNet/ResNet on 700 and 1000 image datasets	Final massive-scale training runs completed and logged
5	25 Jun 2026	Comparative analysis of training metrics across all scaling phases	Identification of the optimal production-ready model
6	26 Jun 2026	Weekly review, metric presentation, and repository commit phase	Finalized data visualizations and pushed source code

5. Methodology

The methodology for Week 5 was characterized by iterative implementation and empirical validation.

Iterative Development (Computer Vision): The Hand Tracking module was built using an iterative, component-based approach. The methodology prioritized establishing a flawless, low-latency video pipeline first, completely isolated from any machine learning logic. Only once the video feed was stable was the complex MediaPipe tensor processing integrated. This isolation prevents compounding errors during debugging.

Scientific Method in Deep Learning: The methodology for the dataset scaling was strictly scientific. The hypothesis being tested was that increasing dataset volume reduces generalization error. The experiment involved training three distinct architectures across four increasing dataset sizes, while maintaining constant hyperparameters (learning rate, batch size, optimization algorithm) across comparable runs. The analysis quantitatively compared validation metrics to prove or disprove the hypothesis and select the optimal deployment candidate.

Data-Driven Decision Making: The selection of MobileNetV2 as the final production model was not arbitrary or based on intuition; it was a methodological conclusion derived directly from the empirical tracking matrices, balancing raw accuracy against computational efficiency.

6. Tools and Technologies Used

Deep Learning Framework	TensorFlow / Keras	Core engine for building, compiling, and executing the massive-scale training runs.
Computer Vision Core	OpenCV (cv2)	Used for hardware camera interfacing, while loop frame grabbing, and color space management (BGR to RGB).
Machine Learning Pipeline	Google MediaPipe	Employed specifically for its solutions.hands API to execute palm detection and 3D landmark regression.
Data Manipulation	Pandas / NumPy	Utilized for organizing the experimental training metrics and handling image arrays.
Data Visualization	Matplotlib / Seaborn	Used to generate the epoch-by-epoch loss curves and the comparative scaling graphs.
Development Environment	Google Colab / VS Code	Colab provided the GPU compute for scaling experiments; VS Code was used for real-time OpenCV script execution.
Version Control	Git / GitHub	Maintained a strict version history of all new scripts and finalized Jupyter Notebooks.

7. Results and Observations

Week 5 produced highly tangible, critical results that represent the culmination of weeks of preparation.

Successful Overfitting Mitigation: The primary observation from the dataset scaling experiments is that volume solves variance. The models trained on 1000 images demonstrated smooth, asymptotic learning curves, definitively solving the acute overfitting problem that plagued the earlier iterations.

Identification of the Optimal Model: Through rigorous empirical comparison, MobileNetV2 trained on the 1000-image dataset was definitively identified as the superior model. It provides the optimal intersection of high validation accuracy (greater than 92%) and low computational latency, making it the prime candidate for web deployment.

Functional Real-Time Landmark Extraction: The foundational Hand & Gesture Python script was successfully deployed. It reliably captures video, executes the MediaPipe models, and renders the 21-point kinematic topology on screen in real-time (sustaining greater than 30 FPS), proving the technical viability of the Human Behavior Tracking module.

8. Challenges Faced

Computational Resource Management: Executing 12 deep learning training runs (specifically the ResNet50 architecture on 1000 images) required careful management of GPU compute time and RAM limits. Handling Out-Of-Memory (OOM) errors during the largest batch loads required precise tuning of the `batch_size` parameter.

Color Space Mismatches: A significant initial hurdle in the OpenCV/MediaPipe integration was the silent failure of the detection model due to passing BGR frames to an RGB-trained network. Debugging this required a deep understanding of how OpenCV handles image matrices in memory versus the TensorFlow-backed MediaPipe expectations.

Managing Massive Data Outputs: Aggregating, cleaning, and visualizing the vast amount of logging data generated by the 12 training sessions (Accuracy, Loss, Validation metrics across hundreds of epochs) was complex and required rigorous spreadsheet management to prevent data corruption or misinterpretation.

9. Learning Outcomes

The heavy implementation focus of Week 5 resulted in substantial, practical skill acquisition:

Real-time Computer Vision Pipelines: Gained practical, hands-on experience building low-latency `cv2.VideoCapture` loops and managing real-time frame buffering and rendering.

MediaPipe Integration: Mastered the practical implementation of Google's MediaPipe API. Learned how to instantiate solutions, manage color space conversions, and parse complex, nested multi-hand landmark objects.

Empirical Model Selection: Solidified the ability to act as a data scientist—moving beyond simply training models to empirically analyzing large-scale training logs to make objective, data-driven decisions on architectural deployment.

Resource Tuning: Developed a strong intuition for managing deep learning hyper-parameters (specifically batch size and early stopping) to optimize compute resources when scaling datasets by orders of magnitude.

Bridging CV and ML: Successfully bridged the gap between traditional image processing (OpenCV matrix manipulation) and advanced applied machine learning (MediaPipe regression models) within a single, cohesive script.

10. Conclusion

Week 5 was an incredibly productive and computationally intense period, marked by the successful execution of complex plans across two distinct technological domains. The transition from theoretical planning to aggressive code implementation was seamless and yielded immediate, validating results.

The execution of the dataset scaling strategy for the Pneumonia Detection System was a resounding success. By methodically increasing the dataset to 1000 images, the severe overfitting issues were definitively resolved. The empirical analysis of the resulting metrics clearly identified MobileNetV2 as the optimal architecture, providing a robust, highly accurate model that is now fully ready for the upcoming deployment phase.

Concurrently, the foundational code for the Hand & Gesture Recognition Module was successfully authored and tested. The integration of OpenCV and MediaPipe into a real-time, low-latency tracking pipeline represents the most significant technical milestone for the Human Behavior Tracking System to date.

With a highly accurate, deployment-ready medical model secured and a functional real-time computer vision engine operational, the internship enters its final phases. The upcoming weeks will focus heavily on bridging these backend engines with frontend user interfaces, transitioning these powerful algorithms into usable, interactive software applications.